

CS639 Proposal: Post-trained Coder with ReAct

Team Members

Botao Rui, brui@wisc.edu

Aagam Sogani, asogani@wisc.edu

Motivation

As we approach graduation, we want to work on a project with a strong engineering component, one that showcases a modern MLE stack and remains closely aligned with current industry trends. Post-training and agentic systems are among the hottest topics in recent years. We therefore plan to post-train an open-source 8B-level instruct model into a stronger coder, adapt the model to a ReAct-style format, and build a ReAct framework that enables multi-turn code revision.

Hypothesis

Under the overall pipeline of Model (Base -> SFT (optional) -> GRPO) -> Agentic (Agent SFT -> Agent RL (stretch)), we expect the post-training stage to yield targeted improvements on coding benchmarks (EvalPlus, LiveCodeBench). After agentic training, we expect the model to achieve further gains in coding accuracy through n-turn code revision within the agent framework.

Methodology

Sanity Check

Before training an 8B-level model, we ran preliminary sanity checks on smaller models. In the first batch of experiments, we used Qwen3-1.7B (which has native reasoning ability) as the base model, applied non-reasoning coding SFT, and then performed a GRPO ablation on both the SFT model and the unsupervised base model using harder coding problems. All models were evaluated without a “directly answer” or “enable thinking” prompt. The results are shown below:

Model	HumanEval	HumanEval+	MBPP	MBPP+
Qwen3-1.7B Auto	75.6%	69.5%	76.5%	66.4%
Qwen3-1.7B SFT Auto	65.9%	61.0%	65.6%	56.3%
Qwen3-1.7B SFT/GRPO Auto	72.6%	68.9%	67.2%	57.7%
Qwen3-1.7B GRPO Only Auto	78.7%	72.6%	78.0%	66.4%

In this batch, Qwen3-1.7B GRPO Only Auto achieved the best performance, while non-thinking SFT caused a regression in coding ability. The experiments in <https://arxiv.org/html/2509.26601v3> likewise show that Qwen3-4B suffers a performance drop under non-thinking SFT, so we believe the non-thinking SFT route is no longer suitable. In the second batch of experiments, we used Qwen2.5-Coder-1.5B-Instruct as the baseline model to examine whether a model without native reasoning ability could learn a chain-of-thought format through Thinking SFT and thereby improve coding performance. The results are shown below:

Model	HumanEval	HumanEval+	MBPP	MBPP+
Qwen2.5-Coder-1.5B-Instruct	57.9%	53.0%	59.5%	51.3%
Qwen2.5-Coder-1.5B-Instruct Thinking SFT 10k Auto	56.1%	48.8%	53.4%	47.4%
Qwen2.5-Coder-1.5B-Instruct Thinking SFT 50k Auto	53.7%	49.4%	53.4%	46.3%
DeepSeek-R1-Distill-Qwen-1.5B Auto (36/32k)	63.4%	59.1%	56.9%	49.7%

These results show that both 10k and 50k Thinking SFT on Qwen2.5-Coder-1.5B-Instruct also degrade the baseline model. We speculate that 1.5B parameters are insufficient to robustly support thinking behavior. One counterexample is DeepSeek-R1-Distill-Qwen-1.5B, which surpasses Qwen2.5-Coder-1.5B-Instruct on HumanEval under autonomous reasoning; however, it was trained on the non-instruct Qwen2.5-1.5B-Math base with 800k distilled examples. Since both the base model and the data scale differ substantially from our case, we do not consider it directly applicable to our setting.

Model Selection

Qwen3-8B

We ultimately choose Qwen3-8B as the final scaled base model, and we expect a direct GRPO-only route to be the best choice, for the following reasons:

1. Qwen3-8B has native thinking ability, so direct GRPO training is highly feasible. The 1.7B experiments already show a clear performance gain. The paper at <https://arxiv.org/html/2510.01624v1> also suggests that, in some cases, applying RL after an SFT-improved model can underperform relative to doing RL directly on the base model.
2. For the initially considered Qwen2.5-Coder-7B-Instruct, many papers report mixed outcomes after SFT: some show gains, while others show regressions. We are more inclined to view Thinking SFT as a method that may raise the final performance ceiling after RLVR for non-thinking models, but whose gains are not stably realized during the initial SFT stage. Only after RL can one clearly assess whether SFT truly improves final performance. For example, <https://arxiv.org/pdf/2603.29957v1> reports that base + SFT + RL is stronger than base + RL; however, in our project, fully validating a 7B model through the entire SFT + RL route would be time-consuming and costly.

GRPO with A100 80G (Rent from Runpod)

Data

OpenCodeInstruct: high-quality all-Python data; we keep examples with multiple unit tests and a length under 4k.

KodCode-light-rl-10k: a harder dataset specialized for RL; we sample 1k examples.

Data cleaning follows AceCoder (<https://arxiv.org/abs/2502.01718>). In GRPO, our procedure is: first let the base model run through the entire dataset, generating 16 attempts per problem; then, offline select problems where $0 < n \leq 4$ attempts succeed as RL data; finally, drop batches with reward = 0 during training.

The total amount of data is about 5k.

Hyperparameters

Setting	Value
Training recipe	LoRA, bfl6
num_generations	8
max_seq	6k
max_response	4k
LoRA rank	64
Max steps	800, with checkpoints per 100 steps
Learning rate	1e-6

Reward Function

Code execution reward * 2; format reward * 0.5.

Scaffold

ms-swift: the official Qwen scaffold for post-training.

Agent SFT with A100 40G (On TACC Lonestar 6)

Data

We use 1k-3k distilled Agent Trace examples in the following format (fix turn varies):

```
<input>...</input>
<think>...</think>
<submit>...</submit>
<obs>status: fail ...</obs>
<think>...</think>
<submit>...</submit>
<obs>status: pass ...</obs>
```

Our distillation procedure follows SCoRe (Student-Centered one-step Reinforcement) (<https://arxiv.org/pdf/2509.14257>). We first let Qwen3-8B-RL solve problems on the coding dataset and collect all failed cases from Qwen3-8B-RL. For these failed cases, we then ask a teacher-model API to continue from the failure state and generate the next correct fix over 1-3 turns, thereby obtaining distilled Agent Traces that successfully repair the code in 1-3 turns.

Hyperparameters

Setting	Value
LoRA rank	64
Context	16k-32k
Learning rate	5e-5
Epochs	3

Agent RL (Stretch)

This component will be explored only if we finish the above pipeline and still have enough time or extra rented compute. The details remain to be refined. One current idea is to do short-horizon RL for repair tasks.

For example, we may provide the model with the following context:

```
<input>problem...</input>
<submit>code_v1</submit>
<obs>fail_1</obs>
```

and ask it to generate the next repair step:

```
<think>...</think>
<submit>...</submit>
```

We then execute the generated code and measure whether the step directly passes all tests, increases the number of passed tests, or at least compiles successfully.

We may also allow the following setup:

Start from a verified fail state:
<input>problem...</input>
<submit>code_v1</submit>
<obs>fail_1</obs>
repair 1 -> environment returns obs -> repair 2 -> environment returns obs
optimize only the final two steps at most.

Evaluation

Model	HumanEval	MBPP	HumanEval+	MBPP+	LiveCodeBench
Qwen3-8B					
Qwen3-8B + RL					
Qwen3-8B + RL + ReAct SFT (3-turn)					
Qwen3-8B + RL + ReAct SFT/RL (Stretch)					
Other Baseline Models for Comparison...					